Computer Science Revision Notes

C. Irving

Contents

1 Programming **1**
- 1.1 Variables . . . 1
- 1.2 Input/Output . . . 3
- 1.3 Comments . . . 4
- 1.4 Selection . . . 5
- 1.5 Iteration . . . 6
- 1.6 Arrays . . . 7
- 1.7 Records . . . 8

2 Data Structures **9**

3 Algorithms **11**

4 Computation **13**

5 Data Representation **15**
- 5.1 Number Systems . . . 15
  - 5.1.1 Casual Definitions . . . 15
  - 5.1.2 Formal Definitions . . . 16
- 5.2 Binary Numbers . . . 18
  - 5.2.1 Binary to Denary Conversion . . . 18
  - 5.2.2 Denary to Binary Conversion . . . 18
  - 5.2.3 Special Properties of Binary Numbers . . . 19
  - 5.2.4 Overflow . . . 19
  - 5.2.5 Fixed-Point Numbers . . . 19
- 5.3 Binary Arithmetic . . . 20
  - 5.3.1 Bit Shifts . . . 20
  - 5.3.2 Binary Addition . . . 20
- 5.4 Sound . . . 21
- 5.5 Images . . . 22
- 5.6 Compression . . . 23
- 5.7 Encryption . . . 24

6 Computer Systems **25**
6.1 Hardware and Software 25
6.2 Operating Systems 26
6.3 Programming Language Classification 27
6.3.1 Machine Code 27
6.3.2 Assembly 27
6.3.3 High-Level Languages 28
6.3.4 Interpreted Languages 29
6.3.5 Compiled Languages 29

7 Organisation and Architecture **31**

8 Consequences of Use **33**

9 Communication and Networking **35**

10 Databases **37**

11 Big Data **39**

12 Functional Programming **41**

13 Problem Solving **43**

Chapter 1

Programming

1.1 Variables

A variable is a named location in memory that can store a value. Variables have data types, which determines what type of data can be stored in them, e.g. strings or integers. Variables can also be set with a value, which tells the program what to store in the variable. Giving a variable an initial value before it is used is called initialisation.

Variables in VB.Net are declared using
`Dim <variable name> As <datatype>`:

```
' Declare a variable caled 'foo' with type String
Dim foo As String

' Declare a variable called 'bar' with type String and
' initialise it with a value of "Hello World"
Dim bar As String = "Hello World"

' Declare a variable called 'baz' with type Integer and
' initialise it with a value of 0
Dim baz As Integer = 0
```

In C++, they are declared using `<datatype> <variable name>;`:

```
// Declare a variable called 'foo' with type std::string
std::string foo;

// Declare a variable called 'bar' with type std::string and
// initialise it with a value of "Hello World"
std::string bar = "Hello World";
```

```
// Declare a variable called 'baz' with type int and
// initialise it with a value of 0
int baz = 0;
```

After declaring a variable, its value can be changed. In VB.Net, this is done using `<variable name> = <new value>`, and in C++ this is done using `<variable name> = <new value>;`.

Calculations can also be done using variables, for example, this is how to add two variable in VB.Net:

```
Dim a As Integer = 2
Dim b As Integer = 5
Dim sum As Integer = a + b ' Add a and b and store the sum
```

And this is it in C++:

```
int a = 2;
int b = 5;
int sum = a + b; // Add a and b and store the sum
```

Other operations such as addition, subtraction, multiplication and division can be done using `+`, `-`, `*` and `/`. As well as these, there is also the modulo and integer divison operations.

Modulo (`Mod` in VB.Net, `%` in C++) takes the first number, divides it by the second number, then takes the remainder. For example, $7 \mod 3 = 1$, because $7 \div 3 = 2$ remainder 1.

Integer divison (\ in VB.Net) takes the first number and divides it by the second number, but rounds down so that it only takes the integer part. It does not have a specific symbol in C++, so it done by `Math.floor(a / b)`, using the `Math` library for the floor function.

1.2 Input/Output

Input and output are how the program can talk to the user. Input takes in data that the user enters, and output gives data back to the user.

Taking an input in VB.Net is done using `foo = Console.ReadLine()`, and in C++, it is done using `std::cin >> foo;`.

Outputting text in VB.Net is done using `Console.WriteLine(bar)`, and in C++, it is done using `std::cout << bar;`.

Using input and output together can take data in, transform it somehow, and then output it. This an example in VB.Net:

```
' Prompt the user to enter their name
Console.WriteLine("Enter your name")

' Take the user's input and store it to a variable called 'name'
Dim name As String = Console.ReadLine()

' Output Hello and the name that they input.
Console.WriteLine("Hello " & name)
```

This is the same example in C++:

```
// Prompt user to enter their name
// \n leaves a new line after the input
std::cout << "Enter your name" << \n;

// Take the user's input and store it to a variable called 'name'
std::string name;
std::cin >> name;

// Output Hello and the name that they input.
std::cout << "Hello " + name;
```

The two examples have to join strings together, which is called concatenation. In VB.Net, this is done using `&`, and in C++, it is done using `+`. This creates a new string made up of the other two. For example, concatenating `"Hello "` and `"World"` would give the string `"Hello World"`. Note that the `"Hello "` string has a space on the end, meaning that there is a space between the two words in the concatenated string.

1.3 Comments

1.4 Selection

1.5 Iteration

1.6 Arrays

1.7 Records

Chapter 2

Data Structures

Chapter 3

Algorithms

Chapter 4

Computation

Chapter 5

Data Representation

5.1 Number Systems

5.1.1 Casual Definitions

Numbers can be categorised into many different groups, starting with the natural numbers. The natural numbers are the numbers used for counting, 0, 1, 2,.... The set of nautral numbers is written:

$$\mathbb{N} = \{0, 1, 2, \dots\} \tag{5.1}$$

The next set is the set of integers ($\mathbb{Z}$). This is the set of all real numbers without a decimal part, including all natural numbers (0, 1, 2,...), and all of the negatives of the natural numbers (-1, -2,...).

$$\mathbb{Z} = \{0, 1, -1, 2, -2, \dots\} \tag{5.2}$$

After the integers, the rational numbers can be defined as the set of all ratios $\frac{a}{b}$, where a and b are integers, and $b \neq 0$. The set of rational numbers is written $\mathbb{Q}$:

$$\mathbb{Q} = \left\{ \frac{a}{b} : a, b \in \mathbb{Z}, b \neq 0 \right\} \tag{5.3}$$

The set of real numbers ($\mathbb{R}$) is the set of all numbers in the interval $(-\infty, \infty)$, i.e. the whole number line.

Each of these sets are proper subsets of each other, with $\mathbb{N}$ being the smallest, and $\mathbb{R}$ being the largest. A subset is defined such that is A is a subset of B ($A \subseteq B$), then B contains every element in A, and A only contains elements in B. Proper subsets are subsets such that $A \subseteq B$ and $A \neq B$, and is written $A \subsetneq B$.

$$\mathbb{N} \subsetneq \mathbb{Z} \subsetneq \mathbb{Q} \subsetneq \mathbb{R} \tag{5.4}$$

Because of how subsets are defined, if $A \subset B \subset C$, then $A \subset C$ ($\subset$ referring to either $\subseteq$ or $\subsetneq$).

As well as these, there are some other notable sets:

The set of all positive integers is written $\mathbb{Z}^+$, which is the same as the set of natural numbers excluding 0[1]:

$$\mathbb{Z}^+ = \mathbb{N} \setminus \{0\} \tag{5.5}$$

The empty set is written $\emptyset$:

$$\emptyset = \{\} \tag{5.6}$$

Since the empty set contains no elements, any set contains every element of $\emptyset$, and $\emptyset$ does not contain any elements not in any given set, so $\emptyset \subseteq A$ for any set A.

The irrational numbers is the set of all real numbers that aren't rational:

$$\text{The set of irrational numbers} = \mathbb{R} \setminus \mathbb{Q} \tag{5.7}$$

There is no widely-accepted symbol for the set of irrational numbers, so $\mathbb{R} \setminus \mathbb{Q}$ is often used.

5.1.2 Formal Definitions

These definitions are **not** required for A-Level.

The Natural Numbers

We will define 0 as the empty set, $0 = \{\}$. Now, we will define the successor of any natural number n as $S(n) = \{n, \{n\}\}$. This means that the successor of 0 is $S(0) = \{0, \{0\}\} = \{\{\}, \{\{\}\}\}$. In casual terms, $S(n) = n + 1$, so $S(0)$ is the definition of 1 in the natural numbers. Now, the set of natural numbers is defined as the set containing 0 such that for all elements $n \in \mathbb{N}$, $S(n) \in \mathbb{N}$. This means that $S(0) \in \mathbb{N}$, which means that $S(S(0)) = S(1) \in \mathbb{N}$, etc.

The Integers

Every integer can be written as a pair of natural numbers, (n_1, n_2) such that $\mathbb{Z} \ni x = n_1 - n_2$.

Proof. For $x \in \mathbb{N}$, x can be written as $x - 0$, so $x = (x, 0)$. For the negative numbers, the opposite can be used; for some integer $x < 0$, x must be a natural number, since $-x$ would be a positive integer if x is a negative integer, and all positive integers are also natural numbers. Thus, x can be written as $0 - (-x)$, so $x = (0, -x)$. Because all natural numbers and all negative integers can be written in this form, that means that all integers can be written in this form ($\mathbb{N} \cup \mathbb{Z}^- = \mathbb{Z}$). □

[1] The symbol $\setminus$ means excluding, i.e. $A \setminus B$ is the set containing all elements of A that are not in B or, formally: $A \setminus B = \{x \in A : x \notin B\}$.

However, this definition means that there are duplicate integers, (e.g. $2-1 = 3-2 = 1$), so we will define an equivalence relation between the integers.

Let $\sim$ be an equivalence relation such that $(a,b) \sim (c,d)$ iff[2] $a-b = c-d$. This means that $(a,b) \sim (c,d)$ iff $a+d = b+c$.

Addition in the natural numbers can be defined as follows:

$x+0 = x$ for any natural number x. For any $x, S(y) \in \mathbb{N}$, $x+S(y) = S(x+y)$.

From here, the integers will fall into equivalence classes (sets of integers such that each integer in a set is equal to all other integers in the set). The proof that $\sim$ is an equivalence relation is as follows:

Proof. By the definition of an equivalence relation: a relation $R \subseteq A^2$ is an equivalence relation iff $R(x,x) \,\forall\, x \in A$ and $R(x,y) \iff R(y,x) \,\forall\, x,y \in A$ and $R(x,y) \wedge R(y,z) \implies R(x,z) \,\forall\, x,y,z \in A$ (where $R(x,y)$ means $(x,y) \in A^2$).

Therefore if $\sim$ is an equivalence relation, for all $x,y,z \in \mathbb{Z}$: $x \sim x$, $x \sim y \iff y \sim x$ and $(x \sim y) \wedge (y \sim z) \implies (x \sim z)$.

- Criterion 1, $x \sim x$: let $x = (a,b)$, thus $x \sim x$ must be true, since $x \sim x \iff a+b = a+b$, which is clearly true.
- Criterion 2, $x \sim y \iff y \sim x$: let $x = (a,b)$ and $y = (c,d)$, thus $x \sim y$ means that $a+d = b+c$, and $y \sim x$ means that $d+a = c+b$. Since addition is commutative, $a+d = d+a = b+c = c+b$.
- Criterion 3, $(x \sim y) \wedge (y \sim z) \implies (x \sim z)$: let $x = (a,b)$, $y = (c,d)$ and $z = (e,f)$. $x \sim y$ can be rewritten as $a+d = b+c$, $y \sim z$ can be rewritten as $c+f = d+e$ and $x \sim z$ can be rewritten as $a+f = b+e$. Thus, if $a+d = b+c$ and $c+f = d+e$, then the two equations can be combined to make $(a+d)+(c+f) = (b+c)+(d+e)$. Removing the c and d term from each side then gives $a+f = b+e \implies x \sim z$.

Thus, all criteria hold, so $\sim$ is an equivalence relation. □

From this equivalence relation, the set of integers can be defined as the set of equivalence classes under $\sim$.

The Rationals

The Reals

[2] 'iff' means if and only if.

5.2 Binary Numbers

Binary numbers are numbers written in base-2, in contrast to the normal denary form (base-10). This means that binary has two digits, 0 and 1. Place value still works like denary, but with powers of 2 instead of powers of 10. This means that the rightmost digit has a place value of 2^0, the next one left has a place value of 2^1, the next one has a place value of 2^2, etc. Binary digits are also called bits.

5.2.1 Binary to Denary Conversion

Working out the value of a binary number means adding together all of the place with 1 digits (nothing has to be multiplied by the digit, since the only digits are 0 and 1, so anything with a 0 is just ignored, and anything with a 1 is added). For example, the denary value of the number 01011010_2 (the $_2$ means base-2) can be calculated as follows:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	1	0	1	0

Add each column with a 1 in: $2^6 + 2^4 + 2^3 + 2^1 = 64_{10} + 16_{10} + 8_{10} + 2_{10} = 90_{10}$ ($_{10}$ means base-10).

So $01011010_2 = 90_{10}$.

5.2.2 Denary to Binary Conversion

To convert back, find the greatest power of two smaller than the number you're converting. Using 90_{10} as an example, $2^7 = 128$ is greater then 90, but $2^6 = 64$ is smaller, so 2^6 is the greatest power of 2 less than 90. Now, subtract 2^6 from 90, and record a 1 in the 2^6 place of the binary number. Repeat with this process, until the number remaining minus the greatest power of two less than it = 0.

$$90 - 2^6 = 26$$
$$26 - 2^4 = 10$$
$$10 - 2^3 = 2$$
$$2 - 2^1 = 0$$

So the columns with 1s are the 2^6, 2^4, 2^3 and 2^1 columns, and the rest are 0s:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	1	0	1	0

So $90_{10} = 01011010_2$.

5.2.3 Special Properties of Binary Numbers

The largest number that can be stored in an n-digit binary number is found by setting all of the digits to 1. For example, the largest value that can be stored in 8 bits is 11111111_2, which is $2^7 + 2^6 + \cdots + 2^0 = 255$. This will always be 1 less than the place value of the $n + 1$th digit, since adding 1 to the number would force every bit to carry, adding a new bit on the left. Therefore, the largest value that can be stored in n bits is $2^n - 1$.

The furthest left and furthest right bits are called the most and least significant bits (MSB/LSB) respectively. The MSB (furthest left) has the greatest place value in the number, and it determines the greatest number that can be represented. The LSB (furthest right) has the least place value in the number ($2^0 = 1$), and this determines the parity of the number (whether it is odd or even). Since all places above the LSB are even (since they are all powers of 2), only the LSB can make the number odd, thus if the LSB is 1, the number is odd, and if it is 0, the number is even.

5.2.4 Overflow

5.2.5 Fixed-Point Numbers

5.3 Binary Arithmetic

Doing arithmetic with binary numbers is very similar to doing arithmetic with denary numbers, just with place values that are powers of 2, rather than powers of 10.

5.3.1 Bit Shifts

A bit shift is where all of the bits in a binary number are moved left or right, with zeros placed appropriately to preserve place value.

Left Shifts

A left shift moves all of the bits left by 1 bit, adding a zero to the furthest right bit (which would otherwise be left empty). Moving all of the bits left by 1 multiplies them all by 2, meaning that the overall effect is multiplying the number by 2. This is equivalent to mulitplication by 10 in denary, where all of the digits are moved left, and a zero is added at the end.

As an example of a left bit shift, this is $90_{10} = 01011010_2$ shifted left by 1 bit:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	1	0	1	0	0

Converting that back to denary gives $10110100_2 = 180_{10}$, meaning that the original 90 has been multiplied by 2.

Right Shifts

Similar to how a left shift multiplies a binary number by two, a right shift divides it by 2. Using integers, this is integer division, i.e. taking $3_{10} = 11_2$ and shifting it right by 1 gives $1_2 = 1_{10}$, not 1.5_{10}. However, if the number is also using decimal bits, the shift will move the LSB to the $2^{-1} = \frac{1}{2}$ bit.

As an example of a left bit shift, this is $90_{10} = 01011010_2$ shifted right by 1 bit:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	1	0	1	1	0	1

Converting that back to denary gives $10110100_2 = 45_{10}$, meaning that the original 90 has been divided by 2.

5.3.2 Binary Addition

5.4 Sound

5.5 Images

5.6 Compression

5.7 Encryption

Chapter 6

Computer Systems

6.1 Hardware and Software

6.2 Operating Systems

6.3 Programming Language Classification

6.3.1 Machine Code

Machine code is the binary 1s and 0s that enter a computer's CPU and control what it does. Machine code is rarely written by humans, since it is very difficult to read and write, however the first programs, written as holes on punch cards were written directly in machine code.

6.3.2 Assembly

Assembly was created as a more readable form of machine code. Assembly uses opcodes and operands to make instructions.

An opcode is a 3-letter mnemonic, short for a command, such as 'store' (`STR`). The opcode is then given operands, which are pieces of data that the command uses, for example, `STR` takes a register `Rn` and a memory reference, where it will store the contents of the `n`th register. To store the value in register 2 into memory location 300, assembly would use the line:

```
STR R2 300
```

The full instruction set for AQA assembly language can be found here. This is an example of a program in assembly:

```
// Compute the 10th Fibonacci number
LDR R0 #0   // R0 will be the counter to check how many
            // iterations have happened
LDR R3 #0   // R1 is the first number to add; it will be
            // shifted into R3
LDR R1 #1   // R2 is the second number to add

fibonacci:          // A label to start the loop
    CMP R0 #9       // Check whether more than 9 iterations
                    // have happened yet
    BGT exit        // If yes, exit
    LDR R2 R1       // Shift R1 into R2
    LDR R1 R3       // Shift R3 (previous sum) into R1
    ADD R3 R1 R2    // Adds the two previous numbers
    ADD R0 R0 #1    // Increments the counter
    B fibonacci     // Loops back to the fibonacci label

exit:               // Exit label for when the program
                    // finishes
    STR R3 100      // Stores the result to memory location
                    // 100
    HALT            // Stop the program
```

Assembly has a one-to-one correspondance with machine code. This means that it can be converted, one instruction mapping to one machine code instruction. For example, the command `ADD R3 R1 R2` could be assembled into the machine code 0011 0011 0001 0010; the opcode (`ADD`) assembling to 0011, and the registers assembling to 0011 (`R3`), 0001 (`R1`), and 0010 (`R2`). This makes it very easy to convert into machine code, hence many early programs were written in it, but it makes it hard to read, even if it is easier than pure machine code. However, machine code gives the programmer complete control over the memory, down to each bit. This means that programs can be optimised perfectly for the CPU, and leave no wasted memory, whereas running code through higher-level languages can cause unnecessary machine code being added.

Each processor has its own assembly instruction set, which will be converted to machine code that that processor understands. This means that assembly can only work on one type of processor, whereas high-level languages will work for any processor that has a suitable interpreter/compiler for it.

Overall, these qualities make assembly good for highly specific programs that run on one chip, but have to run very quickly. For example, an integrated system might use programs written in assembly, since they will waste little to no memory, will only have to run on that one chip, and have to run efficiently.

6.3.3 High-Level Languages

A high-level language is a language that looks more like English than machine code or assembly. The first high-level languages were languages such as Fortran, created by John Backus to look more like English. The program above to compute the 10th Fibonacci number looks like this in Fortran:

```
! Compute the 10th Fibonacci number
program fibonnaci
    integer :: n, i, a, b, sum  ! Declare all variables

    n = 10      ! Initisalise max iteration to 10
    a = 0       ! Set the first number to 0
    b = 1       ! Set the second number to 1

    do i = 2, n         ! Iterate from i = 2 to i = n
        sum = a + b     ! Add a to b
        a = b           ! Shift b to a
        b = sum         ! Shift sum to b
    end do

    print *, sum        ! Output the final sum
end program fibonnaci
```

This shows how much easier to read high-level languages are over low-level languages such as assembly. Variables mean that values can have meaningful

names rather than just numbers, and structures such as iteration (`do i = 2, n`) allow programs to be much shorter by handling multiple assembly commands in one line.

High-level languages mean that programs are easier to code, decreasing production time and making the code more maintainable for future programmers.

6.3.4 Interpreted Languages

Some high-level languages are interpreted, meaning that they are read by an interpreter program line by line, and executed line by line as they are read. This means that programs have to be reread every time they are executed, which will slow them down. An example of an interpreted language is Python.

Interpreted languages require reinterpreting every time they are run, meaning that they can only be run if the person running it has an the interpreter and the source code, which means that the code can be easily copied by anyone with access to it. This makes it not ideal for large programs that are being sold.

6.3.5 Compiled Languages

Compiled languages are read in their entirety by a compiler, which compiles the source code (the code written by the developer) into a file of obeject code (machine code) called an executable. This can be directly run by a CPU, meaning that it is quick to run. However, any small edit to the code requires a full recompile, meaning that development can be slow for large projects that need a lot of recompiles. Some examples of compiled languages are VB.Net, C++, Java and Fortran.

Compiled code can be shared easily as an executable, meaning that the source code doesn't have to be shared as well as the program, making it ideal for a lot of application software.

Chapter 7

Organisation and Architecture

Chapter 8

Consequences of Use

Chapter 9

Communication and Networking

Chapter 10

Databases

Chapter 11

Big Data

Chapter 12

Functional Programming

Chapter 13

Problem Solving